

WEEK – 5

TASK 1 : To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
ramya@LAPTOP-OPDAF309: ~$ nano task1.c
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>
#include <errno.h>

char previous_dir[PATH_MAX] = "";

int main()
{
    char current_dir[PATH_MAX];
    char path[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(previous_dir, current_dir);

    printf("Initial directory: %s\n", current_dir);

    printf("Enter directory to change to: ");
    scanf("%s", path);

    if (chdir(path) == 0)
    {
        printf("Directory changed successfully.\n");

        if (getcwd(current_dir, sizeof(current_dir)) != NULL)
            printf("Current directory: %s\n", current_dir);

        printf("Previous directory: %s\n", previous_dir);
    }
}
```

```

else
{
    printf("Error: Cannot change directory to '%s'\n", path);
    printf("Reason: %s\n", strerror(errno));
}

printf("\nTesting previous directory...\n");

if (chdir(previous_dir) == 0)
{
    if (getcwd(current_dir, sizeof(current_dir)) != NULL)
        printf("Returned to previous directory: %s\n", current_dir);
}
else
{
    perror("chdir");
}

return 0;
}
|

```

```

ramya@LAPTOP-OPDAF309:~/task1$ nano task1.c
ramya@LAPTOP-OPDAF309:~/task1$ gcc task1.c -o task1
ramya@LAPTOP-OPDAF309:~/task1$ ./task1
Initial directory: /home/ramya/task1
Enter directory to change to: /tmp
Directory changed successfully.
Current directory: /tmp
Previous directory: /home/ramya/task1

Testing previous directory...
Returned to previous directory: /home/ramya/task1
ramya@LAPTOP-OPDAF309:~/task1$ ./task1
Initial directory: /home/ramya/task1
Enter directory to change to: /abc/xyz
Error: Cannot change directory to '/abc/xyz'
Reason: No such file or directory

Testing previous directory...
Returned to previous directory: /home/ramya/task1
ramya@LAPTOP-OPDAF309:~/task1$ |

```

TASK 2 : TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
ramya@LAPTOP-OPDAF309: ~$ nano task2.c
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

void execute_external(char **args)
{
    pid_t pid = fork();

    if (pid == 0)
    {
        execvp(args[0], args);
        printf("Invalid command: %s\n", args[0]);
        exit(1);
    }
    else if (pid > 0)
    {
        wait(NULL);
    }
    else
    {
        perror("fork");
    }
}
```

```

void execute_command(char **args)
{
    if (args[0] == NULL)
        return;

    if (strcmp(args[0], "cd") == 0)
    {
        if (args[1] == NULL)
            printf("cd: directory required\n");
        else if (chdir(args[1]) != 0)
            perror("cd");
    }
    else if (strcmp(args[0], "pwd") == 0)
    {
        char cwd[1024];

        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("%s\n", cwd);
    }
    else if (strcmp(args[0], "help") == 0)
    {
        printf("Built-in commands:\n");
        printf("cd    - Change directory\n");
        printf("pwd   - Display current directory\n");
        printf("help  - Display help\n");
        printf("exit  - Exit shell\n");
        printf("Other commands are executed as external commands.\n");
    }
    else if (strcmp(args[0], "exit") == 0)
    {
        printf("Exiting shell...\n");
        exit(0);
    }
}

```

```

    else
    {
        execute_external(args);
    }
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    TASK 2 MINI SHELL\n");
    printf("=====\n");
    printf("Type 'help' for built-in commands.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        int count = 0;

        char *token = strtok(input, " \t\n");

        while (token != NULL && count < MAX_ARGS - 1)
        {
            args[count++] = token;
            token = strtok(NULL, " \t\n");
        }

```

```

        args[count] = NULL;

        execute_command(args);
    }

    return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/task1$ mkdir ../task2
ramya@LAPTOP-OPDAF309:~/task1$ cd ../task2
ramya@LAPTOP-OPDAF309:~/task2$ pwd
/home/ramya/task2
ramya@LAPTOP-OPDAF309:~/task2$ nano task2.c
ramya@LAPTOP-OPDAF309:~/task2$ gcc task2.c -o task2
ramya@LAPTOP-OPDAF309:~/task2$ ./task2
=====
TASK 2 MINI SHELL
=====
Type 'help' for built-in commands.
Type 'exit' to quit.

myshell$ help
Built-in commands:
cd - Change directory
pwd - Display current directory
help - Display help
exit - Exit shell
Other commands are executed as external commands.
myshell$ pwd
/home/ramya/task2
myshell$ cd /tmp
myshell$ pwd
/tmp
myshell$ cd /home/ramya/task2
myshell$ pwd
/home/ramya/task2
myshell$ ls
task2 task2.c
myshell$ echo Hello
Hello
myshell$ date
Sat Sep 19 08:20:31 UTC 2026
myshell$ abcd123
Invalid command: abcd123
myshell$ exit
Exiting shell...
ramya@LAPTOP-OPDAF309:~/task2$ |
```

TASK 3 : To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
ramya@LAPTOP-OPDAF309: ~$ nano task3.c
GNU nano 8.7.1 task3.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>

void save_state()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        FILE *fp = fopen("shell_state.txt", "w");

        if (fp != NULL)
        {
            fprintf(fp, "Last working directory: %s\n", cwd);
            fclose(fp);

            printf("State saved successfully.\n");
        }
        else
        {
            perror("fopen");
        }
    }
    else
    {
        perror("getcwd");
    }
}
```

```
void cleanup()
{
    printf("Cleaning up resources...\n");
    save_state();
    printf("Cleanup completed.\n");
}

int main()
{
    char cwd[PATH_MAX];
    char command[100];

    printf("=====\n");
    printf("      TASK 3 DIRECTORY TOOL\n");
    printf("=====\n");

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("Current directory: %s\n", cwd);
    }
    else
    {
        perror("getcwd");
        return 1;
    }
}
```

```
while (1)
{
    printf("\nEnter command (pwd/exit): ");
    scanf("%s", command);

    if (strcmp(command, "pwd") == 0)
    {
        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("Current path: %s\n", cwd);
        else
            perror("getcwd");
    }
    else if (strcmp(command, "exit") == 0)
    {
        printf("Exit request received.\n");
        cleanup();
        printf("Program exiting...\n");
        break;
    }
    else
    {
        printf("Invalid command.\n");
    }
}

return 0;
}
```

```

ramya@LAPTOP-OPDAF309:~/task2$ mkdir ../task3
ramya@LAPTOP-OPDAF309:~/task2$ cd ../task3
ramya@LAPTOP-OPDAF309:~/task3$ pwd
/home/ramya/task3
ramya@LAPTOP-OPDAF309:~/task3$ nano task3.c
ramya@LAPTOP-OPDAF309:~/task3$ nano task3.c
ramya@LAPTOP-OPDAF309:~/task3$ gcc task3.c -o task3
ramya@LAPTOP-OPDAF309:~/task3$ ./task3
=====
TASK 3 DIRECTORY TOOL
=====
Current directory: /home/ramya/task3

Enter command (pwd/exit): pwd
Current path: /home/ramya/task3

Enter command (pwd/exit): hello
Invalid command.

Enter command (pwd/exit): exit
Exit request received.
Cleaning up resources...
State saved successfully.
Cleanup completed.
Program exiting...
ramya@LAPTOP-OPDAF309:~/task3$ cat shell_state.txt
Last working directory: /home/ramya/task3
ramya@LAPTOP-OPDAF309:~/task3$

```

TASK 4 : To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality give the commands for this tasks and give everything in the same terminal

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int valid_name(const char *name)
{
    if (name == NULL || name[0] == '\\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

void process_export(char *input)
{
    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("Invalid syntax. Use: export NAME=value\\n");
        return;
    }
}
```

```

    *equal = '\0';

    char *name = input;
    char *value = equal + 1;

    if (!valid_name(name))
    {
        printf("Invalid variable name: %s\n", name);
        return;
    }

    if (setenv(name, value, 1) != 0)
    {
        perror("setenv");
        return;
    }

    printf("Variable exported successfully.\n");
    printf("%s=%s\n", name, getenv(name));
}

```

```

void test_child_process(const char *name)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        char *value = getenv(name);

        printf("Child process: %s=%s\n",
               name,
               value ? value : "NOT FOUND");

        exit(0);
    }
    else
    {
        wait(NULL);
        printf("Parent process: Child completed.\n");
    }
}

```

```

int main()
{
    char input[200];
    char name[100];

    printf("=====\n");
    printf("      TASK 4 EXPORT PROGRAM\n");
    printf("=====\n");

    while (1)
    {
        printf("\nEnter command: ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting program...\n");
            break;
        }

        if (strncmp(input, "export ", 7) == 0)
        {
            process_export(input + 7);
        }
        else if (strncmp(input, "check ", 6) == 0)
        {
            strcpy(name, input + 6);

            if (valid_name(name))
            {
                char *value = getenv(name);

                if (value != NULL)
                    printf("%s=%s\n", name, value);
                else

```

```

        printf("%s is not set.\n", name);
    }
    else
    {
        printf("Invalid variable name.\n");
    }
}
else if (strncmp(input, "child ", 6) == 0)
{
    strcpy(name, input + 6);

    if (valid_name(name))
        test_child_process(name);
    else
        printf("Invalid variable name.\n");
}
else
{
    printf("Invalid command.\n");
    printf("Use: export NAME=value\n");
    printf("Use: check NAME\n");
    printf("Use: child NAME\n");
    printf("Use: exit\n");
}
}

return 0;
}

```

```
ramya@LAPTOP-OPDAF309: ~  × + ▾  
ramya@LAPTOP-OPDAF309:~/task4$ nano task4.c  
ramya@LAPTOP-OPDAF309:~/task4$ gcc task4.c -o task4  
ramya@LAPTOP-OPDAF309:~/task4$ ./task4  
=====
```

TASK 4 EXPORT PROGRAM

```
=====
```

Enter command: export NAME=Ramya
Variable exported successfully.
NAME=Ramya

Enter command: check NAME
NAME=Ramya

Enter command: export NAME=Student
Variable exported successfully.
NAME=Student

Enter command: export COLLEGE=BTECH
Variable exported successfully.
COLLEGE=BTECH

Enter command: export 123NAME=Hello
Invalid variable name: 123NAME

Enter command: export MY-NAME=Hello
Invalid variable name: MY-NAME

Enter command: export STUDENT=Ramya
Variable exported successfully.
STUDENT=Ramya

Enter command: child STUDENT
Child process: STUDENT=Ramya
Parent process: Child completed.

Enter command: check UNKNOWN
UNKNOWN is not set.

Enter command: exit
Exiting program...